



PERTEMUAN VI

CLUSTERING

6.1 TUJUAN PEMBELAJARAN

- A. Mahasiswa memahami konsep dasar *unsupervised learning* dan perannya dalam *machine learning*.
- B. Mahasiswa mampu menerapkan berbagai teknik *clustering* seperti K-Means, K-Medoids, dan DBSCAN untuk mengelompokkan data berdasarkan karakteristik tertentu.
- C. Mahasiswa dapat membandingkan kualitas hasil clustering menggunakan metrik evaluasi seperti *Inertia* dan *Silhouette Score*.
- D. Mahasiswa memahami konsep *Association Rule* dan mampu menerapkannya untuk menemukan pola hubungan antar-item dalam dataset.

6.2 DASAR TEORI

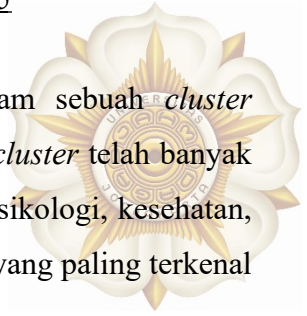
A. Unsupervised Learning

Unsupervised Learning merupakan salah satu pembelajaran mesin dimana model harus menemukan pola dari data tanpa bantuan label eksternal. Pembelajaran ini mengaplikasikan algoritma untuk menganalisis dan menemukan pola dalam data tanpa intervensi atau bantuan manusia. Fokusnya adalah eksplorasi data dan pengelompokan menggunakan algoritma seperti K-Means, *Artificial Neural Network* (ANN), dan Gaussian Mixture Model (GMM) yang menawarkan fleksibilitas yang lebih besar meski dengan akurasi yang biasanya lebih rendah (Nurhalizah & Ardianto, 2024).

Algoritma ini digunakan dalam pendeteksian pola dan pemodelan deskriptif untuk membentuk dasar algoritma untuk menemukan model yang tepat. Algoritma ini digunakan untuk clustering dan aturan asosiasi. Keuntungan *unsupervised learning* adalah algoritma lebih fleksibel untuk mencari pola yang mungkin belum diketahui sebelumnya karena tidak bergantung pada label (Wijoyo, et al., 2024).

B. Clustering

Clustering atau klasterisasi merupakan suatu teknik atau metode untuk mengelompokkan data. *Clustering* adalah salah satu alat yang penting dalam pengolahan data statistik untuk melakukan analisis data. Analisis *cluster* merupakan seperangkat



metode yang digunakan untuk mengelompokkan objek ke dalam sebuah *cluster* berdasarkan informasi yang ditemukan pada data. Saat ini analisis *cluster* telah banyak digunakan di berbagai bidang ilmu pengetahuan seperti ekonomi, psikologi, kesehatan, social masyarakat dan kependudukan. Salah satu metode clustering yang paling terkenal adalah K-Means (Afidaha & Masrukan, 2023).

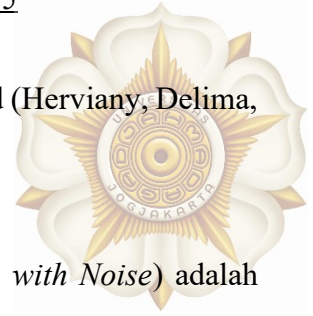
C. K-Means

Algoritma K-Means merupakan algoritma non hirarki yang berasal dari metode data clustering. Algoritma K-Means dimulai dengan pembentukan partisi klaster di awal kemudian secara interaktif partisi cluster ini diperbaiki hingga tidak terjadi perubahan yang signifikan pada partisi cluster. K-Means ini mempartisi data ke dalam kelompok sehingga data berkarakteristik sama dimasukkan kedalam satu kelompok yang sama dan data yang berkarakteristik berbeda dikelompokkan kedalam kelompok yang lain. Prinsip utama dari teknik ini adalah menyusun K buah partisi/pusat massa (centroid)/rata-rata (mean) dari sekumpulan data. Adapun tujuan dari pengelompokan data ini adalah untuk meminimalkan fungsi objektif yang diset dalam proses pengelompokan yang pada umumnya berusaha meminimalkan variasi dalam suatu kelompok dan memaksimalkan variasi antar kelompok (Sulistiyawati & Supriyanto, 2021).

D. K-Medoids

Algoritma K-Medoids berperan untuk menemukan Medoids dalam suatu cluster yang menjadi titik pusat cluster. K-Medoids lebih kuat daripada K-Means seperti pada K-Medoids kami mendapatkan K selaku objek representatif diminimalkan hasil ketidaksamaan objek data, sedangkan K-Means menggunakan hasil kuadrat jarak Euclidean pada data benda. Dan metrik jarak ini mengurangi data yang noise dan outliers. Ini adalah teknik berbasis objek yang representatif. Dalam metode ini kita memilih objek sebenarnya untuk merepresentasikan cluster daripada mengambil nilai rata-rata objek dalam cluster sebagai titik referensi.

Algoritma K-Medoids secara komputasi lebih sulit daripada K-Means karena menghitung medoid menggunakan frekuensi kejadian. K-Medoids memiliki potensi penting karakteristik pusat mana yang berada di antara data tunjuk sendiri. Algoritma baru diusulkan untuk pengelompokan K-Medoids yang berjalan seperti K-Means



algoritma dan menguji beberapa metode untuk memilih awal medoid (Herviany, Delima, Nurhidayah, & Kasini, 2021).

E. DBSCAN

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) adalah algoritma *clustering* berbasis kepadatan yang mengelompokkan titik data berdasarkan seberapa padat area di sekitarnya. Algoritma ini menggunakan dua parameter, yaitu epsilon (ϵ) sebagai radius pencarian dan minPts sebagai jumlah minimum titik agar sebuah titik dianggap core point. DBSCAN unggul dalam mendeteksi cluster dengan bentuk yang tidak beraturan dan dapat mengidentifikasi noise atau outlier tanpa membutuhkan jumlah cluster di awal (Ester et al., 1996; Han, Kamber, & Pei, 2011).

F. Association Rule

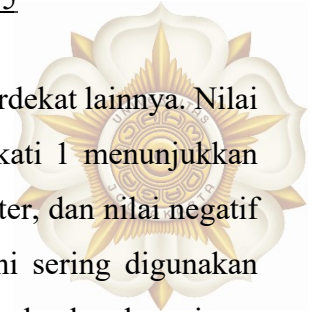
Association Rule Mining merupakan teknik dalam *data mining* untuk menemukan pola atau hubungan antar-item dalam dataset transaksi. Aturan asosiasi biasanya direpresentasikan dalam bentuk $A \rightarrow B$, yang berarti bahwa kemunculan item A sering diikuti oleh kemunculan item B. Penilaian kualitas aturan dilakukan menggunakan *support*, *confidence*, dan *lift*, yang masing-masing mengukur frekuensi, probabilitas keterkaitan, serta kekuatan hubungan antar-item dalam dataset. Metode ini umum digunakan dalam *market basket analysis* untuk mengungkap pola perilaku konsumen (Agrawal, Imieliński, & Swami, 1993; Han, Kamber, & Pei, 2011).

G. Inertia

Inertia adalah metrik evaluasi pada algoritma K-Means yang mengukur tingkat kompaknya sebuah cluster. Nilai inertia dihitung sebagai jumlah kuadrat jarak seluruh data terhadap centroid cluster-nya. Nilai inertia yang lebih rendah menunjukkan cluster yang lebih kompak dan representatif. Metrik ini sangat sering digunakan dalam *Elbow Method* untuk menentukan jumlah cluster optimal, meskipun hanya relevan untuk metode clustering berbasis centroid seperti K-Means (Bishop, 2006; scikit-learn documentation).

H. Silhouette Score

Silhouette Score merupakan metode evaluasi yang mengukur seberapa baik suatu data ditempatkan pada cluster tertentu dengan mempertimbangkan kedekatan data



tersebut terhadap cluster-nya sendiri dan jaraknya terhadap cluster terdekat lainnya. Nilai silhouette berada pada rentang -1 hingga 1 , di mana nilai mendekati 1 menunjukkan pemisahan cluster yang baik, nilai 0 menandakan overlap antar-cluster, dan nilai negatif menunjukkan kemungkinan pengelompokan yang salah. Metrik ini sering digunakan untuk membandingkan beberapa algoritma clustering dalam sebuah eksperimen (Rousseeuw, 1987; Tan, Steinbach, & Kumar, 2019).

6.3 ALAT DAN BAHAN

A. Perangkat Keras

1. Laptop dengan minimal RAM 4 GB
2. Koneksi internet stabil

B. Perangkat Lunak

1. Browser (Chrome, Edge, Firefox, dsb)
2. Google Colab / Jupyter Notebook
3. Library ML (pandas, numpy, scikit-learn, matplotlib, dsb)

6.4 LANGKAH PERCOBAAN

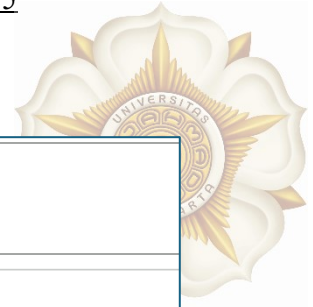
1. Persiapkan IDE

Mahasiswa memulai percobaan dengan membuka Google Colab sebagai lingkungan pengembangan utama. Colab dipilih karena mendukung berbagai *library machine learning* yang diperlukan, seperti *pandas*, *numpy*, *scikit-learn*, *matplotlib*, hingga *sklearn-extra* untuk K-Medoids. Selain itu, Colab menyediakan GPU/CPU gratis sehingga proses komputasi *clustering* dapat berjalan lebih cepat tanpa membebani perangkat mahasiswa.

2. Instalasi dan Impor Library

Setelah lingkungan siap, mahasiswa mengimpor seluruh library yang dibutuhkan untuk melakukan preprocessing, visualisasi, dan proses clustering. Library utama meliputi:

1. *pandas* dan *numpy* untuk manipulasi data
2. *StandardScaler* untuk standarisasi
3. *K-Means* dan *K-Medoids* untuk algoritma clustering
4. *matplotlib* dan *seaborn* untuk visualisasi



5. silhouette_score untuk evaluasi hasil clustering, dsb

```
!pip install kneed
!pip install scikit-learn-extra
!pip install numpy==1.26.4

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn_extra.cluster import KMedoids
from sklearn.metrics import silhouette_score
from kneed import KneeLocator, DataGenerator
from sklearn.metrics import silhouette_samples, silhouette_score
import plotly.express as px
from sklearn.preprocessing import LabelEncoder
```

Gambar 6.4.1 Impor Pustaka

3. Read dan Load Dataset Information

Mahasiswa kemudian memuat dataset Credit Card Dataset for Clustering ke dalam DataFrame. Dataset ini berisi data perilaku transaksi pelanggan kartu kredit, seperti jumlah saldo, total pembayaran, jumlah pembelian, penarikan cash advance, dan pembayaran minimum. Pada tahap ini mahasiswa:

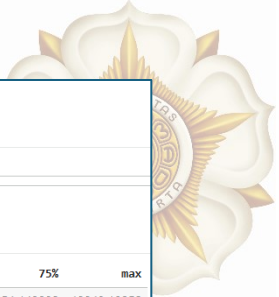
1. Mengimpor dataset dari sumber online atau drive,
2. Menampilkan beberapa baris awal untuk memastikan dataset terbaca dengan benar

```
# load dataset
df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/CC%20GENERAL.csv', delimiter=',')
df.head()
```

	CUST_ID	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_FREQUENCY	ONEOFF_PURCH
0	C10001	40.900749	0.818182	95.40	0.00	95.4	0.000000	0.166667	
1	C10002	3202.467416	0.909091	0.00	0.00	0.0	6442.945483	0.000000	
2	C10003	2495.148862	1.000000	773.17	773.17	0.0	0.000000	1.000000	
3	C10004	1666.670542	0.636364	1499.00	1499.00	0.0	205.788017	0.083333	
4	C10005	817.714335	1.000000	16.00	16.00	0.0	0.000000	0.083333	

Gambar 6.4.2 Membaca dataset

3. Melihat struktur dataset melalui fungsi info() dan describe() untuk memahami jenis data dan distribusinya.



```
# size dataset
df.shape
(8950, 18)

# ringkasan statistik
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
BALANCE	8950.0	1564.474828	2081.531879	0.000000	128.281915	873.385231	2054.140036	19043.13856
BALANCE_FREQUENCY	8950.0	0.877271	0.236904	0.000000	0.888889	1.000000	1.000000	1.000000
PURCHASES	8950.0	1003.204834	2136.634782	0.000000	39.635000	361.280000	1110.130000	49039.57000
ONEOFF_PURCHASES	8950.0	592.437371	1659.887917	0.000000	0.000000	38.000000	577.405000	40761.25000
INSTALLMENTS_PURCHASES	8950.0	411.067645	904.338115	0.000000	0.000000	89.000000	468.637500	22500.00000
CASH_ADVANCE	8950.0	978.871112	2097.163877	0.000000	0.000000	0.000000	1113.821139	47137.21176

Gambar 6.4.3 Melihat struktur Dataset

```
# korelasi antar kolom dataset
df.corr(numeric_only=True)
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES
BALANCE	1.000000	0.322412	0.181261	0.164350	
BALANCE_FREQUENCY	0.322412	1.000000	0.133674	0.104323	
PURCHASES	0.181261	0.133674	1.000000	0.916845	
ONEOFF_PURCHASES	0.164350	0.104323	0.916845	1.000000	
INSTALLMENTS_PURCHASES	0.126469	0.124292	0.679896	0.330622	
CASH_ADVANCE	0.496692	0.099388	-0.051474	-0.031326	
PURCHASES_FREQUENCY	-0.077944	0.229715	0.393017	0.264937	
ONEOFF_PURCHASES_FREQUENCY	0.073166	0.202415	0.498430	0.524891	

Gambar 6.4.4 Melihat korelasi antar kolom

4. Data Cleaning

Dataset kemudian diperiksa untuk memastikan tidak ada data hilang atau nilai yang tidak sesuai. Mahasiswa:

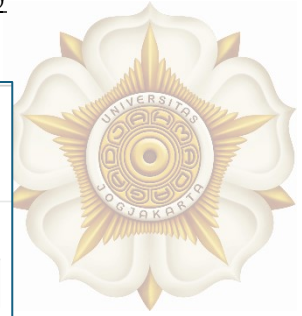
1. Menghapus kolom CUST_ID karena bersifat identitas dan tidak relevan bagi proses clustering

```
df_new = df.drop('CUST_ID', axis = 1)
df_new.head()
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_FREQUENCY
0	40.900749	0.818182	95.40	0.00	95.4	0.000000	
1	3202.467416	0.909091	0.00	0.00	0.0	6442.945483	
2	2495.148862	1.000000	773.17	773.17	0.0	0.000000	
3	1666.670542	0.636364	1499.00	1499.00	0.0	205.788017	
4	817.714335	1.000000	16.00	16.00	0.0	0.000000	

Gambar 6.4.5 Menghapus kolom CUST_ID

2. Mengisi (*impute*) nilai kosong pada fitur seperti MINIMUM_PAYMENTS dan CREDIT_LIMIT menggunakan nilai median agar tidak mengganggu hasil clustering



```
# check missing values
```

```
df_new.isnull().sum()
```

...	0
BALANCE	0
BALANCE_FREQUENCY	0
PURCHASES	0
ONEOFF_PURCHASES	0

Gambar 6.4.6 Menghitung jumlah kolom null

```
# imputasi
```

```
df_new['MINIMUM_PAYMENTS'].fillna(df_new['MINIMUM_PAYMENTS'].median(), inplace=True)
```

```
df_new['CREDIT_LIMIT'].fillna(df_new['CREDIT_LIMIT'].median(), inplace=True)
```

/tmp/ipython-input-1602066474.py:3: FutureWarning:

A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate result is a copy. For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'.

Gambar 6.4.7 Imputasi Data

3. Melakukan pengecekan ulang untuk memastikan seluruh nilai hilang telah teratasi

```
# cek hasil imputasi
```

```
df_new.isnull().sum()
```

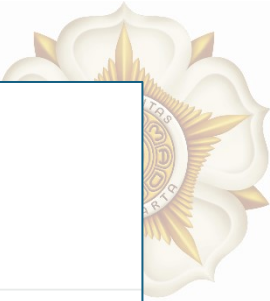
...	0
BALANCE	0
BALANCE_FREQUENCY	0
PURCHASES	0
ONEOFF_PURCHASES	0
INSTALLMENTS_PURCHASES	0

Gambar 6.4.8 Cek hasil Imputasi Data

Pembersihan data penting dilakukan karena nilai kosong dapat menyebabkan algoritma clustering gagal konvergen atau menghasilkan cluster yang tidak akurat.

5. Standarisasi Data

Sebelum melakukan *clustering*, mahasiswa wajib melakukan standarisasi menggunakan *StandardScaler*. Hal ini dilakukan karena sebagian fitur seperti “BALANCE” atau “PURCHASES” memiliki skala yang jauh lebih besar dibanding fitur lain. Dengan standarisasi, setiap fitur akan memiliki distribusi dengan mean = 0 dan standar deviasi = 1 sehingga algoritma clustering tidak bias terhadap fitur berskala besar.



```
# scaling

X = df_new.astype(float).values
scaler = StandardScaler().fit(X)
X_new= scaler.transform(X)
X_new

array([[ -0.73198937, -0.24943448, -0.42489974, ..., -0.3024    ,
        -0.52555097,  0.36067954],
       [ 0.78696085,  0.13432467, -0.46955188, ...,  0.09749953,
        0.2342269 ,  0.36067954],
       [ 0.44713513,  0.51808382, -0.10766823, ..., -0.0932934 ,
        -0.52555097,  0.36067954],
       ...,
       [-0.7403981 , -0.18547673, -0.40196519, ..., -0.32687479,
        0.32919999, -4.12276757],
       [-0.74517423, -0.18547673, -0.46955188, ..., -0.33830497,
        0.32919999, -4.12276757],
       [-0.57257511, -0.88903307,  0.04214581, ..., -0.3243581 ,
        -0.52555097, -4.12276757]])
```

Gambar 6.4.9 Data Scaling

6. Menentukan Jumlah Cluster (Elbow Method & Silhoutte Score)

Untuk menentukan jumlah cluster optimal, mahasiswa melakukan dua tahap evaluasi:

1. Elbow Method

Mahasiswa menjalankan K dengan jumlah cluster 1 hingga 10 dan mencatat nilai inertia. Nilai inertia kemudian divisualisasikan untuk menemukan titik “siku” yang menunjukkan penurunan inertia tidak lagi signifikan.

a. K-Means

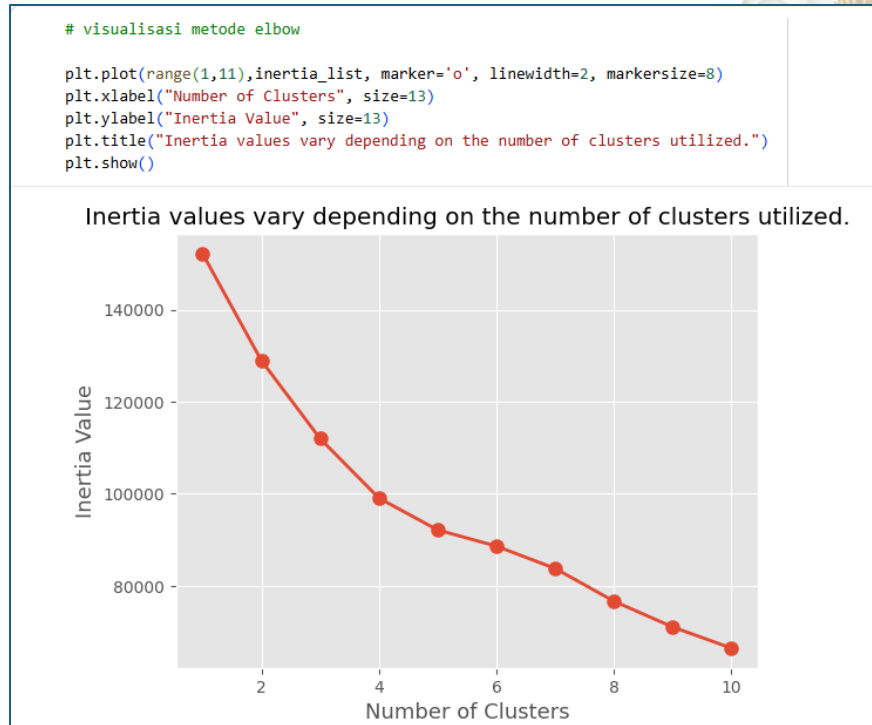
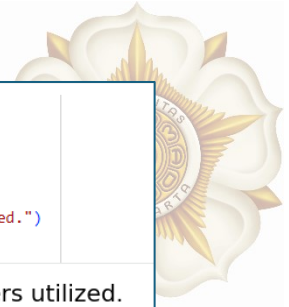
```
from sklearn.cluster import KMeans

# metode elbow untuk menentukan jumlah k

inertia_list = []
for num_clusters in range(1, 11):
    kmeans_model = KMeans(n_clusters=num_clusters)
    kmeans_model.fit(X_new)
    inertia_list.append(kmeans_model.inertia_)
    print("For n_clusters = {}, inertia value is {}".format(num_clusters, kmeans_model.inertia_))

... For n_clusters = 1, inertia value is 152150.00000000004)
For n_clusters = 2, inertia value is 128936.94462472606)
For n_clusters = 3, inertia value is 111973.97335474695)
For n_clusters = 4, inertia value is 99062.37980412853)
For n_clusters = 5, inertia value is 92131.46554452999)
For n_clusters = 6, inertia value is 88621.00610679774)
For n_clusters = 7, inertia value is 83766.7010851524)
For n_clusters = 8, inertia value is 76657.09288658875)
For n_clusters = 9, inertia value is 71051.78747876118)
For n_clusters = 10, inertia value is 66459.87103455774)
```

Gambar 6.4.10 Elbow Method



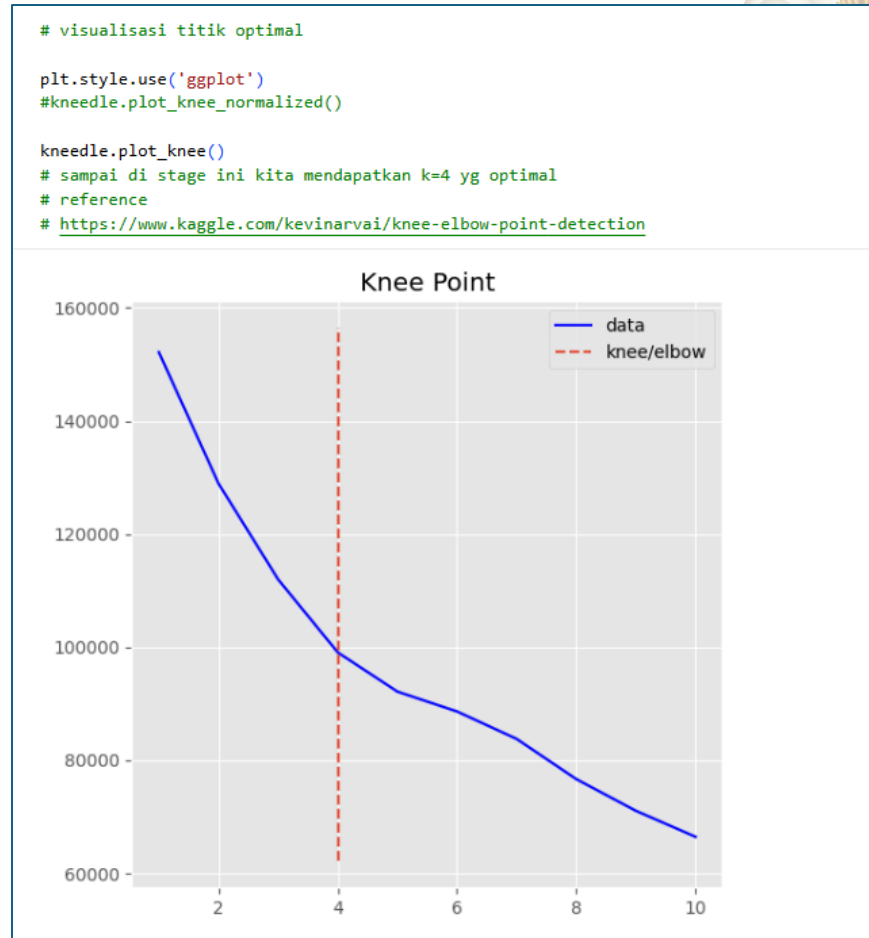
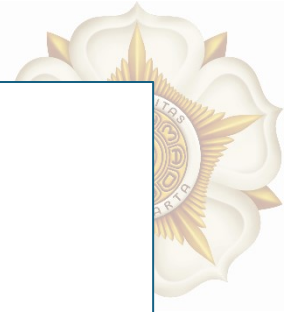
Gambar 6.4.11 Visualisasi Elbow Method

```
# menentukan titik optimal menggunakan kneed

kneedle = KneedleLocator(range(1,11),inertia_list, S=1.0, curve='convex', direction='decreasing')
print(round(kneedle.knee, 3))
print(round(kneedle.elbow, 3))
```

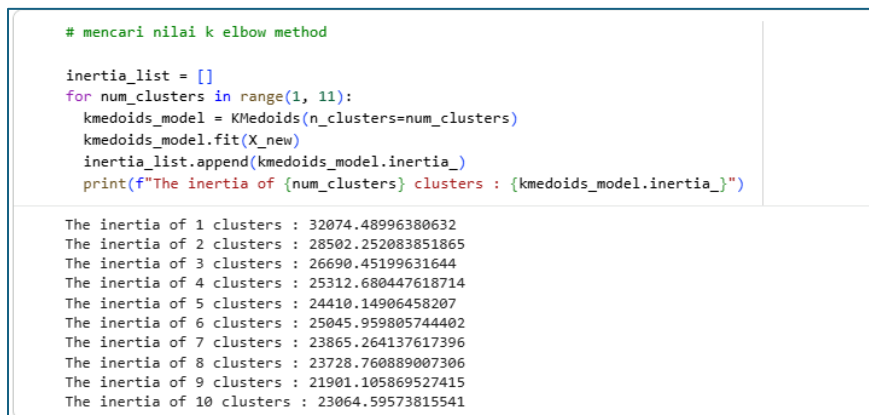
4
4

Gambar 6.4.12 Menentukan titik optimal menggunakan kneed

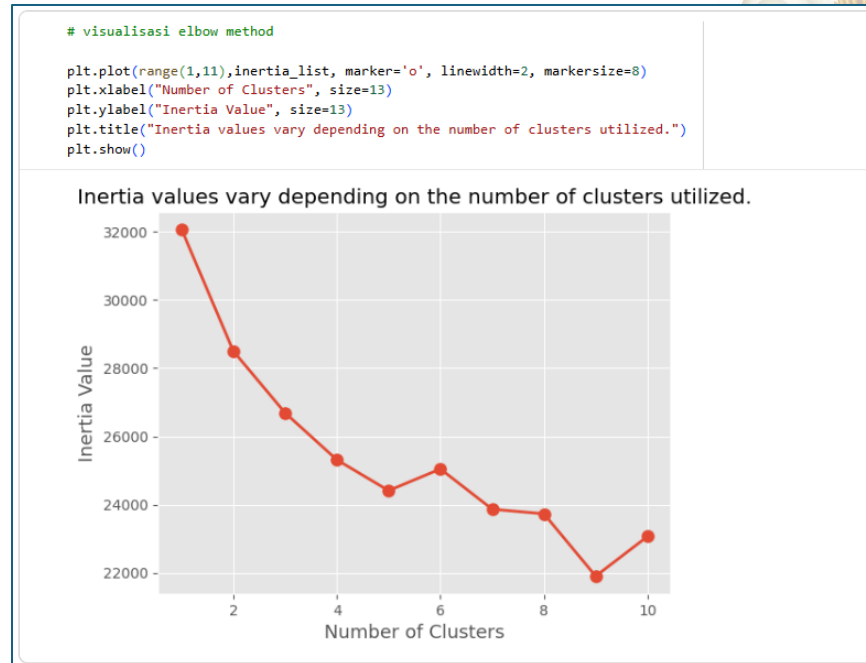
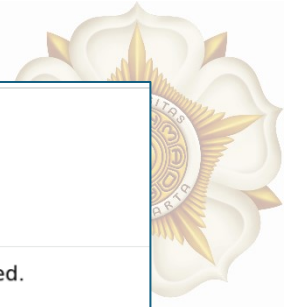


Gambar 6.4.13 Visualisasi Knee Point

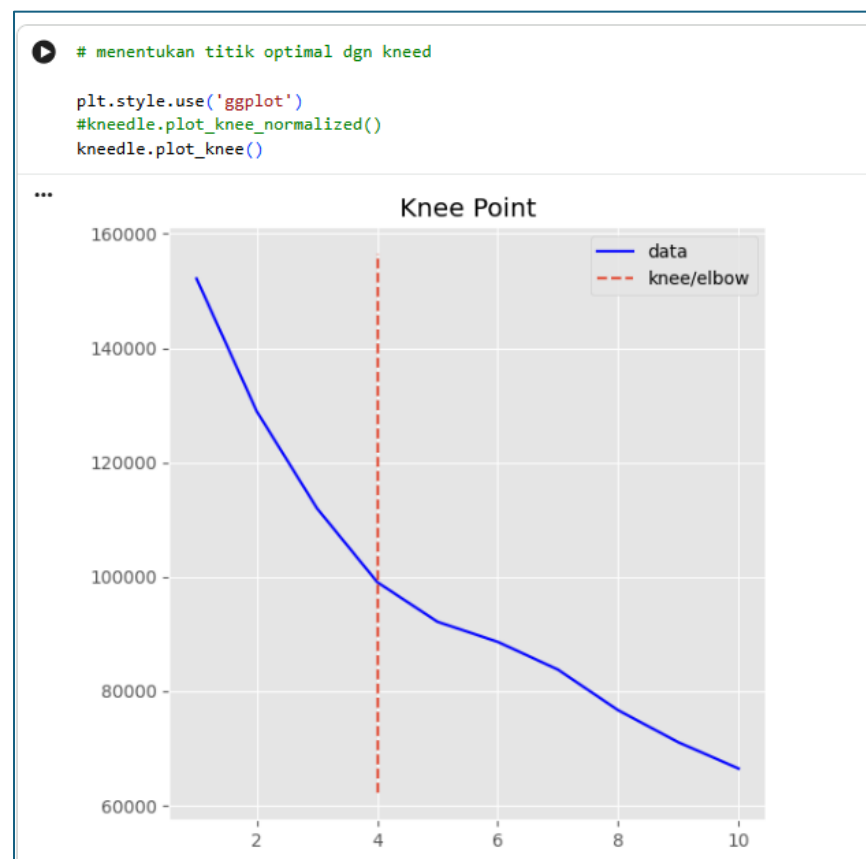
b. K-Medoids



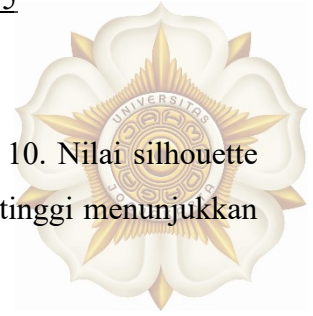
Gambar 6.4.14 Mencari nilai k elbow method



Gambar 6.4.15 Visualisasi elbow method



Gambar 6.4.16 Visualisasi Knee Point



2. Silhouette Score

Mahasiswa menghitung nilai silhouette untuk cluster 2 hingga 10. Nilai silhouette membantu menilai kualitas pemisahan cluster, di mana nilai tertinggi menunjukkan cluster yang paling terpisah dengan baik.

a. K-Means

```
# menentukan silhouette score

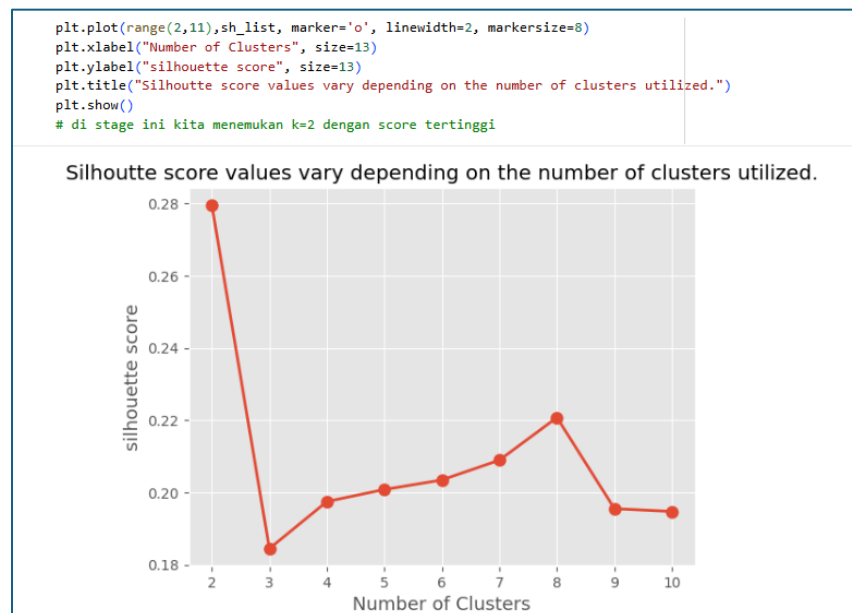
from sklearn.metrics import silhouette_samples, silhouette_score

sh_list = []
for num_clusters in range(2, 11):
    kmeans = KMeans(n_clusters=num_clusters)
    cluster_labels = kmeans.fit_predict(X_new)

    score = silhouette_score(X_new, cluster_labels)
    sh_list.append(score)
    print("For n_clusters = {}, silhouette score is {}".format(num_clusters, score))

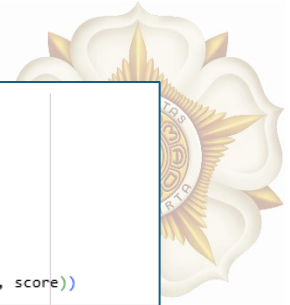
For n_clusters = 2, silhouette score is 0.27953987242960154)
For n_clusters = 3, silhouette score is 0.18447069544775743)
For n_clusters = 4, silhouette score is 0.19749974009976254)
For n_clusters = 5, silhouette score is 0.20083224723760784)
For n_clusters = 6, silhouette score is 0.2034742960849018)
For n_clusters = 7, silhouette score is 0.20896235614091754)
For n_clusters = 8, silhouette score is 0.22075625282182323)
For n_clusters = 9, silhouette score is 0.195520101529126)
For n_clusters = 10, silhouette score is 0.1947488460098652)
```

Gambar 6.4.17 Menentukan silhouette score



Gambar 6.4.18 Visualisasi Solhouette score

b. K-Medoids

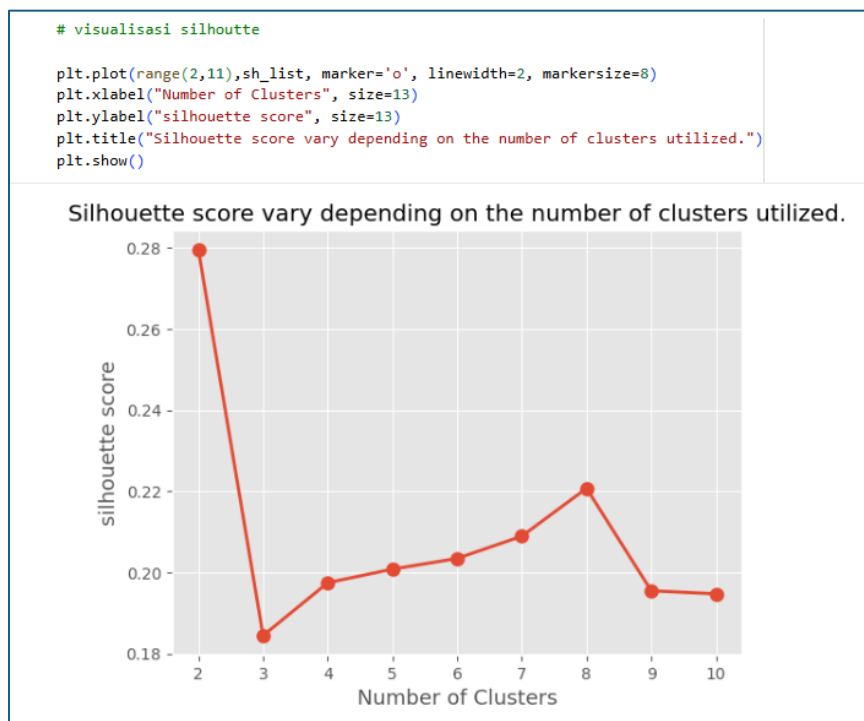


```
from sklearn.metrics import silhouette_samples, silhouette_score
sh_list = []

for num_clusters in range(2, 11):
    kmedoids = KMedoids(n_clusters=num_clusters)
    cluster_labels = kmedoids.fit_predict(X_new)
    score = silhouette_score(X_new, cluster_labels)
    sh_list.append(score)
    print("For n_clusters = {}, silhouette score is {}".format(num_clusters, score))

For n_clusters = 2, silhouette score is 0.1945165537378692)
For n_clusters = 3, silhouette score is 0.16024040458444103)
For n_clusters = 4, silhouette score is 0.13738603460433335)
For n_clusters = 5, silhouette score is 0.14859330798827058)
For n_clusters = 6, silhouette score is 0.07497117420304265)
For n_clusters = 7, silhouette score is 0.051193591100429474)
For n_clusters = 8, silhouette score is 0.03912406632314492)
For n_clusters = 9, silhouette score is 0.08491760468793152)
For n_clusters = 10, silhouette score is 0.03469231089500277)
```

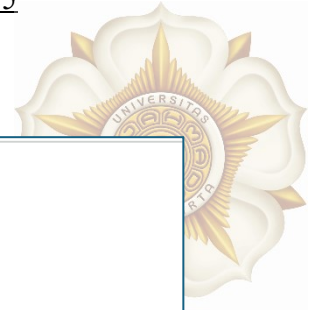
Gambar 6.4.19 Kmedoid



Gambar 6.4.20 Visualisasi Kmedoid

7. Menerapkan Algoritma K-Means

Setelah jumlah cluster ditentukan, mahasiswa menjalankan algoritma K-Means pada data yang telah distandarisasi. Tahap ini meliputi:



1. Melatih model K-Means sesuai jumlah cluster optimal

```
# algoritma kmeans menggunakan nilai silhoutte

k_means = KMeans(n_clusters = 2, random_state = 42)
k_means.fit(X_new)
labels = k_means.labels_
df_new['cluster_labels'] = labels
df_new.head()
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURC
0	40.900749	0.818182	95.40	0.00	
1	3202.467416	0.909091	0.00	0.00	
2	2495.148862	1.000000	773.17	773.17	
3	1666.670542	0.636364	1499.00	1499.00	
4	817.714335	1.000000	16.00	16.00	

Gambar 6.4.21 Training K-Mean sesuai kluster optimal

2. Menghasilkan label cluster untuk setiap data dan menambahkan label cluster ke DataFrame

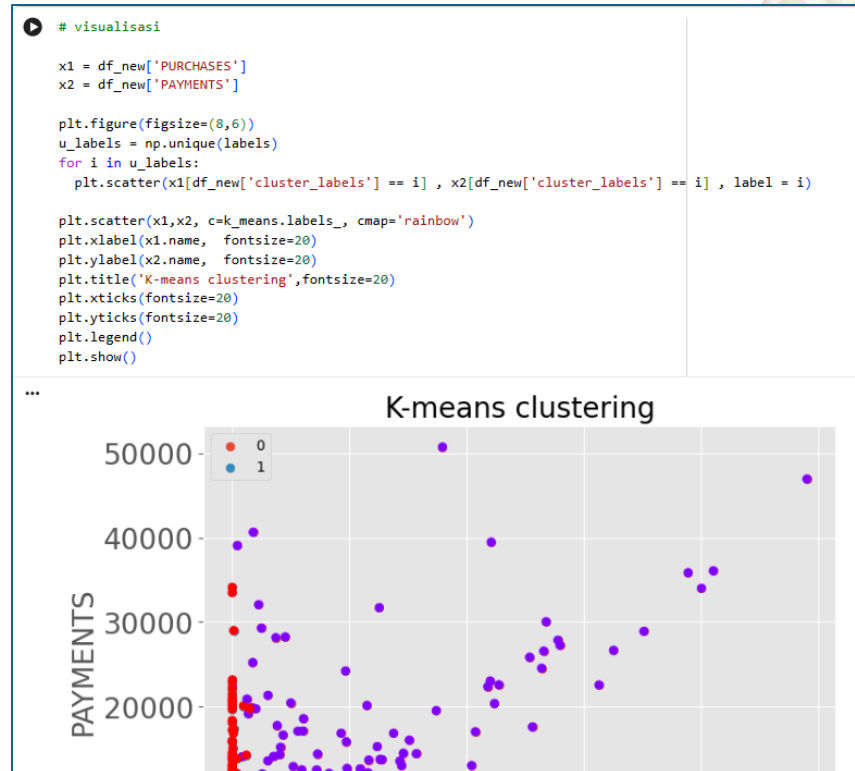
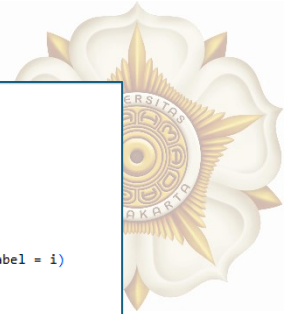
```
# cek centroids

centroids = k_means.cluster_centers_
centroids
```

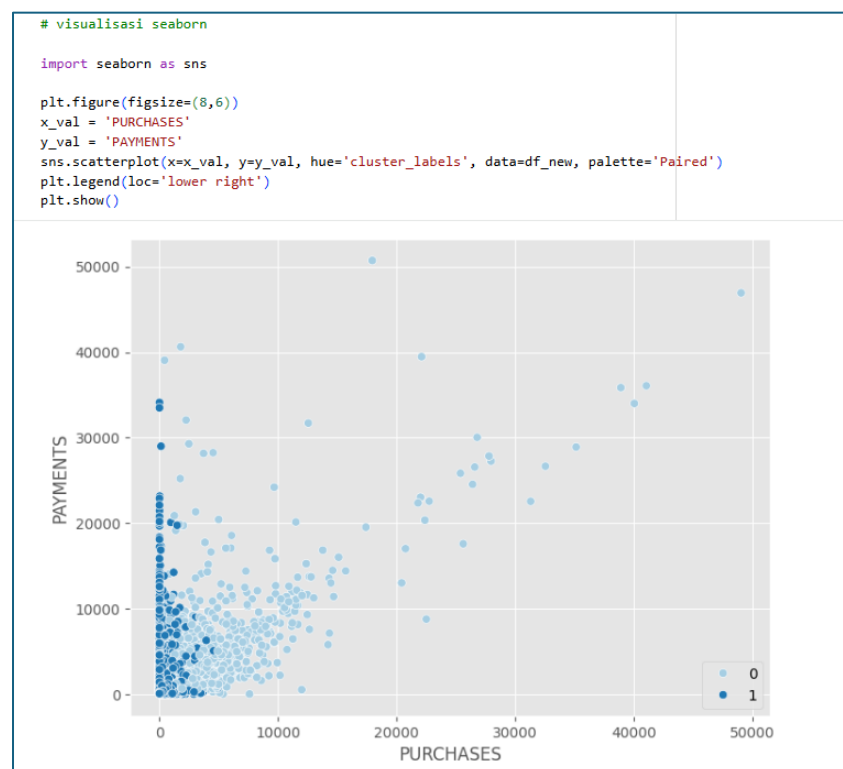
```
array([[ 0.28073957,  0.43247256,  1.10795859,  0.89576982,  0.97358346,
        -0.10307996,  1.1156293 ,  1.29514356,  0.87789229, -0.26387424,
        -0.14839813,  1.26760651,  0.74056992,  0.64489838,  0.15037657,
         0.44987573,  0.27971163],
       [-0.07339984, -0.1130707 , -0.28967769, -0.23420057, -0.25454508,
         0.02695043, -0.29168321, -0.33861752, -0.22952646,  0.06899037,
         0.03879895, -0.33141791, -0.19362328, -0.16860979, -0.03931621,
        -0.11762079, -0.07313109]])
```

Gambar 6.4.22 Cek Centroids

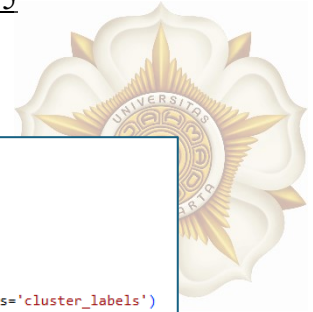
3. Melakukan visualisasi 2D/3D menggunakan fitur-fitur seperti PURCHASES, PAYMENTS, dan BALANCE.



Gambar 6.4.23 Visualisasi K Means Clustering



Gambar 6.4.24 Visualisasi klustering



```
# visualisasi plotly

import plotly.express as px

x_val = 'PURCHASES'
y_val = 'PAYMENTS'
z_val = 'BALANCE'
fig = px.scatter_3d(df_new, x=x_val, y=y_val, z=z_val, color='cluster_labels', labels='cluster_labels')
fig.show()
```

Gambar 6.4.25 Visualisasi Scatter

8. Menerapkan Algoritma K-Medoids

Mahasiswa kemudian mengulangi proses di atas menggunakan K-Medoids, algoritma berbasis medoid yang lebih tahan terhadap outlier dibandingkan K-Means. Langkahnya meliputi:

1. Evaluasi jumlah cluster menggunakan inertia

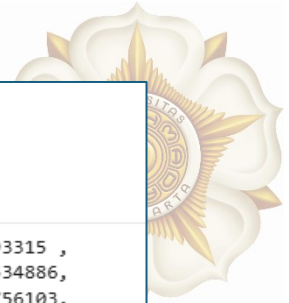
```
# algoritma kmedoids menggunakan elbow

k_medoids = KMedoids(n_clusters = 4, random_state = 42)
k_medoids.fit(X_new)
labels = k_medoids.labels_
df_new['cluster_labels'] = labels
df_new.head()
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE
0	40.900749	0.818182	95.40	0.00	95.4	0.0
1	3202.467416	0.909091	0.00	0.00	0.0	6442.9
2	2495.148862	1.000000	773.17	773.17	0.0	0.0
3	1666.670542	0.636364	1499.00	1499.00	0.0	205.7
4	817.714335	1.000000	16.00	16.00	0.0	0.0

Gambar 6.4.26 Evaluasi jumlah cluster menggunakan inertia

2. Pelatihan model K-Medoids dan menambahkan label cluster ke DataFrame



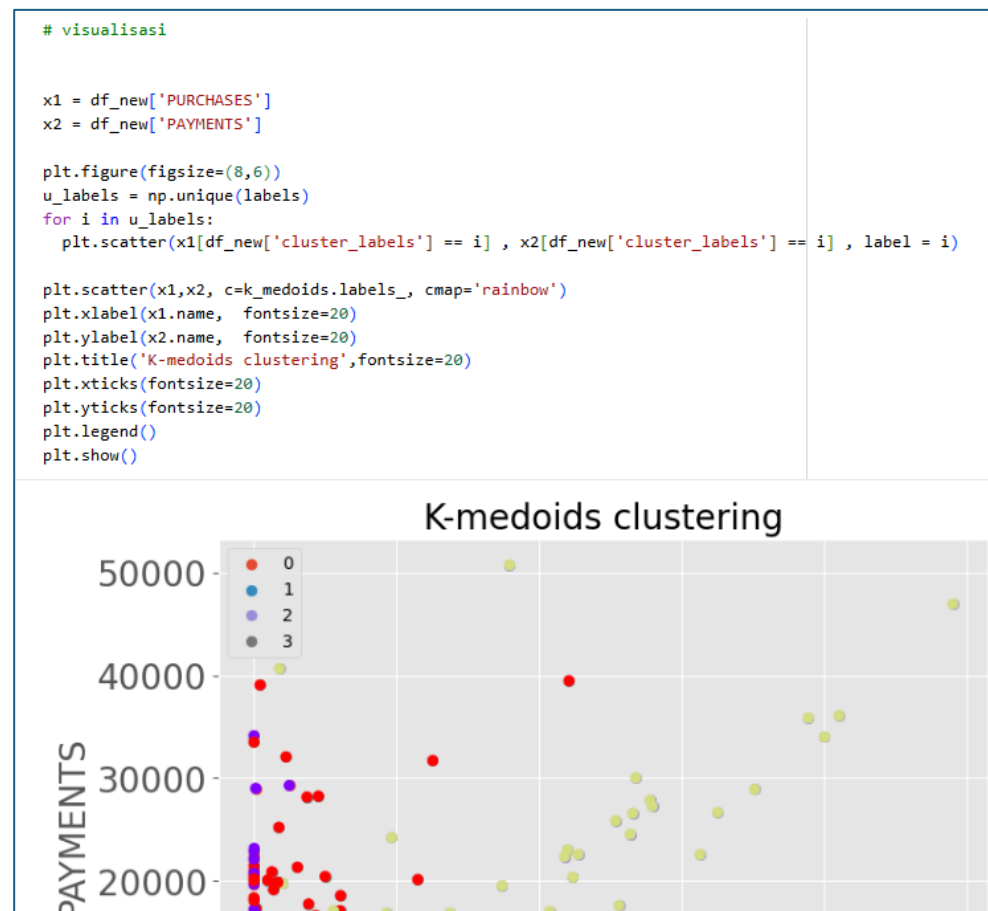
```
# cek centroids

centroids = k_medoids.cluster_centers_
centroids

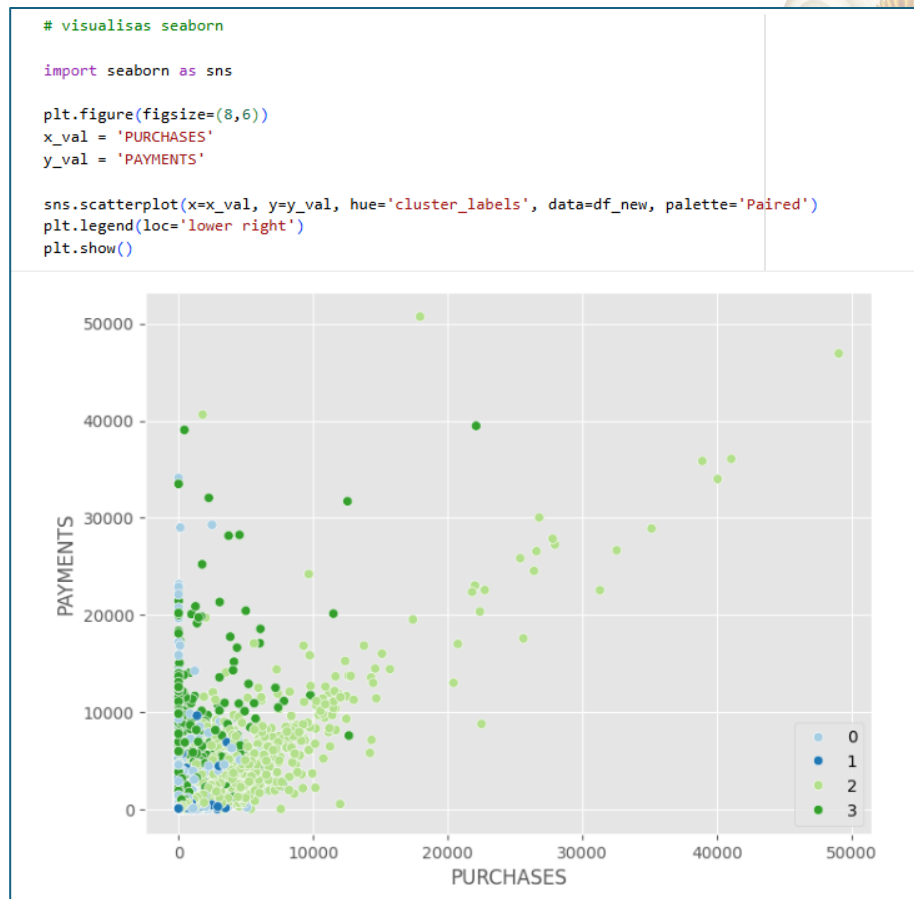
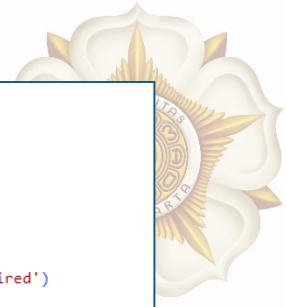
array([[ -0.73017008, -1.78447531, -0.15174467,  0.01660604, -0.3893315 ,
        -0.46678555, -0.39122513, -0.39931927, -0.49762862, -0.67534886,
        -0.47606982, -0.39063931, -0.20456068, -0.35069111, -0.32756103,
         0.50015018,  0.36067954],
       [ -0.04321221,  0.51808382, -0.41993839, -0.29307072, -0.45457623,
        -0.44947139, -1.01412545, -0.39931927, -0.91699519, -0.25891333,
        -0.18299798, -0.51133325, -0.41069279, -0.47801089, -0.17593333,
        -0.52555097,  0.36067954],
       [ -0.65121444,  0.51808382,  0.24224266,  0.03901242,  0.50039651,
        -0.46678555,  1.06221062,  0.15936716,  1.38951716, -0.67534886,
        -0.47606982,  0.37375564, -0.27327139,  0.0131038 , -0.2796588 ,
         0.04428414,  0.36067954],
       [ 1.16410803,  0.51808382, -0.40096824, -0.32982224, -0.3423 ,
         0.80421891, -0.18359002, -0.39931927, -0.2879466 ,  1.8232743 ,
         0.69621752, -0.350408 ,  0.41383563, -0.14385479,  0.17482124,
        -0.52555097,  0.36067954]])
```

Gambar 6.4.27 Pelatihan model K-Medoids

3. Visualisasi hasil clustering untuk membandingkan pola dengan K-Means



Gambar 6.4.28 Visualisasi K-Medoids clustering

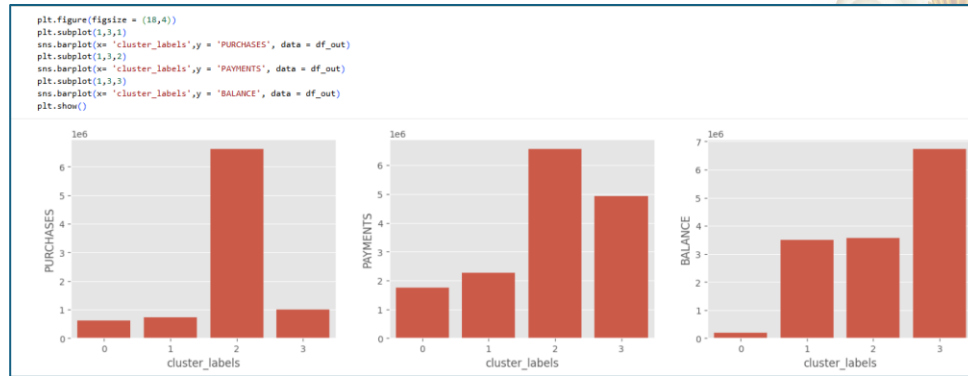


Gambar 6.4.29 Visualisasi Kmedoid Scatter

9. Analisis dan Interpretasi Cluster

Setelah kedua algoritma dijalankan, mahasiswa melakukan analisis pada setiap cluster dengan melihat fitur-fitur seperti:

1. rata-rata saldo
2. total pembelian
3. total pembayaran
4. jumlah transaksi, dan lainnya.

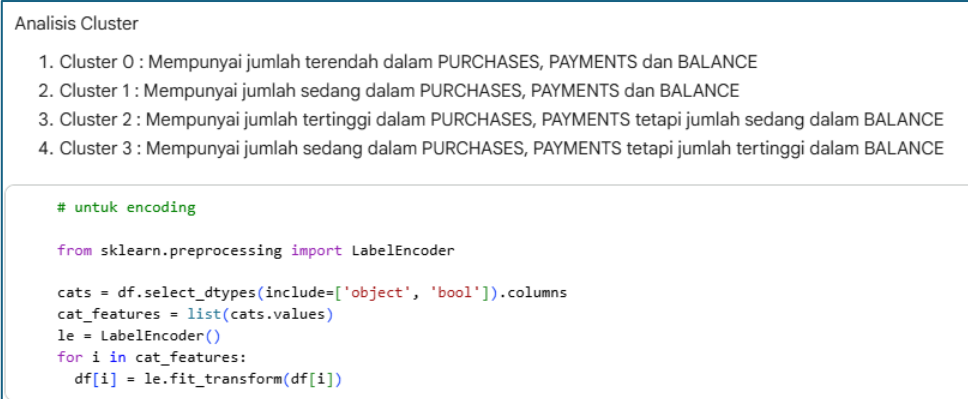


Gambar 6.4.30 Interpretasi Cluster

Mahasiswa kemudian membuat ringkasan atau profil cluster yang menjelaskan karakteristik setiap kelompok, misalnya:

1. cluster dengan pelanggan bertransaksi tinggi
2. cluster dengan pembayaran minimum rendah
3. cluster dengan penggunaan kartu kredit intensif

Analisis ini bertujuan membantu mahasiswa memahami bagaimana algoritma clustering digunakan untuk segmentasi pelanggan.



Gambar 6.4.31 Analisis Cluster

6.5 TUGAS DAN ANALISIS

1. Dataset Preparation

- Download dataset : <https://www.kaggle.com/datasets/arjunbhasin2013/ccdata>
- Upload di google drive atau github masing-masing
- Jelaskan tujuan penggunaan dataset ini
- Jelaskan feature dataset ini



2. Menggunakan dataset lain, silahkan mengerjakan clustering menggunakan K-Means dan K-Medoids. Dataset Air Traffic Passengers Statistics:

https://data.sfgov.org/Transportation/Air-Traffic-Passenger-Statistics/rkru-6vcg/about_data

6.6 REFERENSI

Afidaha, M. W., & Masrukan, M. H. (2023). *Penerapan Metode Clustering dengan Algoritma K-Means untuk Pengelompokkan Data Migrasi Penduduk Tiap Kecamatan di Kabupaten Rembang*. PRISMA, Prosiding Seminar Nasional Matematika (pp. 729-738). Semarang: Jurusan Matematika, Universitas Negeri Semarang.

Agrawal, R., Imieliński, T., & Swami, A. (1993). Mining association rules between sets of items in large databases. *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*, 207–216.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

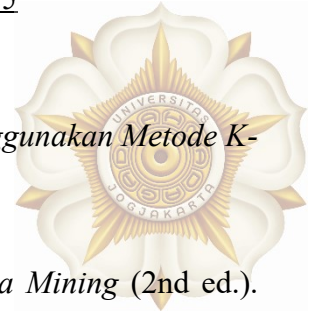
Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, 226–231.

Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.

Herviany, N., Delima, R., Nurhidayah, S., & Kasini, A. (2021). *Perbandingan Algoritma K-Means dan K-Medoids untuk Pengelompokkan Daerah Rawan Tanah Longsor di Provinsi Jawa Barat*. MALCOM: Indonesian Journal of Machine Learning and Computer Science, 34-40.

Nurhalizah, S., & Ardianto, D. (2024). *Penerapan Unsupervised Learning dalam Analisis Pola Data*. Jurnal Sains Komputer dan Informatika.

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.



Sulistiyawati, A., & Supriyanto, E. (2021). *Pengelompokan Data Menggunakan Metode K-Means untuk Analisis Cluster*. Jurnal Matematika dan Statistika.

Tan, P.-N., Steinbach, M., & Kumar, V. (2019). *Introduction to Data Mining* (2nd ed.). Pearson.

Wijoyo, S., et al. (2024). *Unsupervised Learning and Its Application in Pattern Discovery*. Jurnal Teknologi Informasi dan Sains Data.

scikit-learn. (2024). *Clustering Performance Evaluation – Inertia & Silhouette Score*. Diakses dari dokumentasi resmi scikit-learn.